eFactor Backend Requirements — Remote Device State Sync

2026-09-21

Summary

The app cannot show a change made on the physical remote because the cloud never sends per-parameter device state to the phone. This is a backend/firmware gap, not a mobile bug.

The client side is already built and verified in a live session on 21 September 2026 (Redmi 2201117TI, user 707, nodes `...7EC2000000040300` and `...7EC2000000049200`). The app connects to the cloud WebSocket, requests node state every 10 seconds, receives the reply, parses it and repaints bound controls. The data is what is missing: every reply carries device catalogue metadata and no current parameter values.

For a fan whose schema declares five controls, the cloud reports exactly one state field.

Schema control binding	Reported by cloud today
`1_power_command_2_toggle_switch`	Yes, as `devices[].power`
`1_buzzer_command_2_toggle_switch`	No
`1_speed_command_2_slider`	No
`1_direction_command_2_trigger`	No
`1_rpm_command_0_text`	No

> **Request 1 is the blocking item.** Requests 2 to 5 are correctness and performance problems found in the same session.

Evidence

This is a verbatim `getAllNodes` reply captured on the handset at 10:59:12, trimmed to one node. The app had requested state for node `...040300` specifically.


```
{"total":2,"all_nodes":[{
  "config":{
    "node_id":"FFFFFFFFFFFFFFFF7EC2000000040300",
    "node_type":"WIFI",
    "devices":[{
      "id":"FFFFFFFFFFFFFFFF7EC2000000040300",
      "module_type":"WIFI",
      "device_name":"fan_7E000403",
      "name":"fan_7E000403",
      "power":true,
      "ui_type":"Toggle Switch",
      "page_name":"Page1.Operation",
      "ui_description":"Read/Write",
      "local_control":"enable",
      "primary":"power",
      "type":"efactor.device.fan",
      "brand_logo":"https://efactor.tech/files/brand_logo/efactor.jpg",
      "device_image":"https://efactor.tech/files/brand_logo/fan-EF-003FMN03.jpg",
      "Device Communication":"disable",
      "Status":"disable"
    }]
  },
  "role":"primary",
```

```
  "status":{"connectivity":{"connected":"true","timestamp":1789968552000}}
}]}
```

Everything here except `power` and `connected` is static catalogue data: names, image URLs, UI hints. There is no `Buzzer`, no `Speed`, no `Direction`, no `rpm`.

The same reply arrived at 10:59:12, 10:59:22, 10:59:32 and 10:59:42, identical each time, confirming the request/response cycle works and simply returns no state.

The payload that does carry parameters is the `auto_get_nodes_params` telemetry push documented as Issue 6 in the earlier `BACKEND_ISSUES_REPORT.md`. It arrived **zero times** during the session. The full log buffer was searched for it.

Request 1 — deliver per-parameter state (blocking)

The app needs the current value of every parameter the device schema declares. Either option below works; option A is preferred because it makes the UI update immediately rather than on the next poll.

Option A — push on change. When a parameter changes on the device for any reason, including the physical remote, emit an `auto_get_nodes_params` message over the cloud WebSocket to every subscriber of that node. This is the mechanism already described in the previous report; it is simply not firing.

Option B — include state in the query reply. Add a parameters block to the `getAllNodes` and `getAllNodesStatus` replies, so the app's existing 10-second poll picks it up.

Either way the shape should be a flat map of parameter name to current value, per device:

```
{"all_nodes":[{
  "config":{"node_id":"FFFFFFFFFFFFFFFF7EC2000000040300"},
  "parameters":{
    "power": true,
    "Buzzer": false,
    "Speed": 3,
    "Direction": "00",
    "rpm": 9,
    "Current": 0,
    "Voltage": 25.19
  },
  "parameters_updated_at": 1789968552000
}]}
```

The `parameters_updated_at` timestamp matters: the app applies an optimistic value when the user taps and holds it for 12 seconds. A timestamp lets the app resolve a conflict correctly instead of guessing. If a timestamp is impractical, say so and the app will keep its current time-window approach.

Without this change no amount of app-side work can make remote changes visible.

Request 2 — payload shape

Three shape problems. The app works around all three today, but each workaround is a guess that will break on the next payload change.

Problem	Today	Should be
Parameters keyed under an empty string	`"": {"power":"true", "Speed":3}`	`"parameters": {"power":true, "Speed":3}`
Booleans sent as strings	`"connected":"true"`, `"power":"true"`	`"connected":true`, `"power":true`

`node_id` nested inconsistently	`all_nodes[].node_id` in one reply, `all_nodes[].config.node_id` in another	one position, always

The third one caused a real bug. The app looked for `all_nodes[].node_id`; when `getAllNodes` nested it under `config` instead, the id came back empty, the "is this my node?" check passed by default, and **both** fans' values were applied to whichever device page was open. The app now reads both positions, but one consistent position would remove the class of bug.

Also worth noting: `"name": null` and `"name": "NULL"` both appear for the same field across the two nodes in one reply. Please use JSON `null` consistently.

Request 3 — parameter naming contract

The device reports `power`, `Buzzer`, `Speed`. The SDUI schema binds those same controls as `1_power_command_2_toggle_switch`, `1_buzzer_command_2_toggle_switch`, `1_speed_command_2_slider`. Nothing in either payload states that these refer to the same thing.

The app currently bridges the gap by fuzzy matching: it lowercases the telemetry name, splits each schema key on underscores and looks for the name among the parts. That works for `Buzzer` to `1_buzzer_command_2_toggle_switch`, but it is fragile. A telemetry field named `current` would match any schema key containing that word, and a device with two of the same parameter on different pages is ambiguous.

Either of these removes the guesswork:

1. Have telemetry use the same key the schema binds, so `power` becomes `1_power_command_2_toggle_switch`.
2. Or publish the mapping inside the device schema, for example a `telemetryKey` field on each entry of `controls[]`, so the app can resolve it exactly.

Option 2 is less disruptive since it does not change the firmware payload.

Request 4 — command replies should echo the resulting state

`sendData` currently returns an empty payload:

```
sendData CLOUD success code=200 body=

{"payload":""}
```

HTTP 200 with an empty body tells the app the command was accepted by the cloud, not that the device applied it. Returning the resulting parameter value would let the app confirm rather than assume:

```
{"payload":{"node_id":"...040300","parameter":"Buzzer","value":true,"applied_at":1789968552000}}
```

Related: the round trip takes about 4.3 seconds (09:20:17.408 request, 09:20:21.875 reply; a second sample was 3.9 s). Routing logs `sendData routing: CLOUD` even when the protocol is MQTT and the device is on the same LAN as the phone. If LAN routing is meant to be used when mDNS has found the node, it is not engaging.

Request 5 — device schema transfer cost

`api_schema.php` returns 1,898,091 bytes for model `EF-003FMN03` and re-sends all of it on every request, with no way for the app to skip an unchanged body.

Observation	Value
Response size, `EF-003FMN03`	1.9 MB
Response size, `EFCKER V1.0`	6.6 MB
Response size, `pavan-2608`	3.1 MB
`Content-Length` header	absent (`contentLength=unknown`)
`ETag` / `Last-Modified`	absent
Status on an unchanged schema	always `200`, never `304`

In one 3-minute window the same 1.9 MB schema was fetched three times and each sync reported no structural change. That is roughly 5.7 MB of mobile data to learn nothing.

Two asks:

3. Send an `ETag` (or `Last-Modified`) and honour `If-None-Match` / `If-Modified-Since` with `304 Not Modified`. The app already sends `Cache-Control: no-cache` on forced refreshes and has a local cache ready to revalidate.
4. Send a `Content-Length`. Without it the app cannot size its read timeout and falls back to a flat 30 seconds.

This matters beyond bandwidth. If Request 1 cannot be delivered soon, the app's fallback is to poll this schema for parameter values, and at 1.9 MB a response that is not revalidatable makes that fallback impractical.

Lower priority

Mixed HTTP and HTTPS scan endpoints. The `scanApi` published inside the device schema is `https://` for `EF-003FMN03` and `EFCKER V1.0`, but plain `http://` for `pavan-2608`. Android blocks cleartext traffic by default, so the `http://` one will fail on a release build. All published endpoints should be `https://`.

Inconsistent schema versions. `DEVICE_UI_SCHEMA_EXTRACTED` reports `schemaVersion=4.0.0` for `EF-003FMN03` and `EFCKER V1.0`, but `4.1.0` for `pavan-2608`. If 4.1.0 adds or changes fields, please confirm what differs so the client can handle both; if it is only a version bump, aligning them avoids confusion.

Node connectivity reported as offline. At 10:58:16 both nodes reported `"connected":"false"`, then at 10:59:04 node `...040300` reported `"connected":"true"` with no device change in between. Worth confirming the broker heartbeat is reliable, since the app disables controls for a node it believes is offline.

How to verify the fix

No app change is needed to test Request 1. The client already polls and applies whatever arrives.

5. Open the device page for a fan in the app.
6. Press a button on the physical remote, for example the buzzer.

7. The control should change on screen within about 10 seconds.

To confirm from the backend side without the app, subscribe to the cloud WebSocket as user `707` and check that either an `auto_get_nodes_params` push arrives after the remote press, or that the next `getAllNodes` reply contains the changed parameter value.

The app logs every message it receives, so if something arrives and still does not appear on screen, this line names the mismatch and lists the keys the schema expects:

```
[SDUI][REMOTE_SYNC] no schema parameter matched "<name>" (value=<v>).
  Known keys: 1_rpm_command_0_text, 1_buzzer_command_2_toggle_switch,
  1_direction_command_2_trigger, 1_power_command_2_toggle_switch,
  1_speed_command_2_slider
```

Send that line back and the mapping can be corrected on either side.